

Сильно-ленивая куча

Постановка задачи

Хотим реализовать очередь с приоритетом, которая будет делать операции

- `push(x)` — добавить элемент
- `pop()` — удалить минимальный элемент

за оптимальные [теоретически] число сравнений и время работы. То есть хотим, чтобы в худшем случае `push` отработывал за $O(1)$ и сравнений, и времени, а `pop` отработывал за $\lg(n) + O(1)$ сравнений и $O(\lg(n))$ времени. Причем хотим, чтобы дополнительной информации хранилось лишь $O(1)$.

Так, например, биномиальная куча нас не устраивает 1) при `pop` она делает $2 \lg(n) + O(1)$ сравнений; 2) она хранит $O(n)$ дополнительной информации [каждое значение хранится в некоторой ноде, ноды между собой связаны указателями].

План такой: придумаем две новые кучи [работающие где-то очень плохо, где-то оптимально], а потом из них слепим, устраивающую нас, кучу. Операции `push` и `pop` выражаются через функцию `sift_down(i, n)` [в куче размера n только элемент с индексом i нарушает порядок, `sift_down` изменяет кучу так, чтобы порядок стал правильным, «просеивая» элемент $a[i]$ вниз]. Поэтому часто мы будем говорить и про функцию `sift_down`.

Сильная куча

Возьмем бинарную кучу и добавим в нее больше порядка [такую кучу назовем *сильной кучей*]: в правом ребенке лежит всегда значение не меньше, чем в левом [это свойство называется *левой-доминантностью*; порядок тут в обратном порядке, да].

На сильной куче нам будет удобно реализовать два разных алгоритма просеивания.

strengthening_sift_down

Реализуем не самую эффективную по количеству сравнений функцию

`strengthening_sift_down(i, n)`, см. Рис. 1.

```
method left-child(i)
return 2i + 1

method sibling(i)
if i = 0
| return 0
return i + odd(i) - even(i)

method is-leaf(i, n)
if odd(i)
| return sibling(i) ≥ n
return left-child(i) ≥ n

method strengthening-sift-down(i, n)
x ← a[i]
while not is-leaf(i, n)
| j ← sibling(i)
| if even(i)
| | j ← left-child(i)
| else if j < n and left-child(i) < n and
| | a[left-child(i)] ≤ a[j]
| | j ← left-child(i)
| if x ≤ a[j]
| | break
| a[i] ← a[j]
| i ← j
a[i] ← x
```

Рис. 1. Реализация функции `strengthening_sift_down`.

Она работает подобно `sift_down` на обычной бинарной куче, но еще дополнительно следит за тем, чтобы порядок на соседних детях соблюдался.

Лемма 1 `strengthening_sift_down(0, n)` работает за $2\lg(n)$ сравнений и $O(\lg(n))$ времени.

Доказательство. На каждом уровне бинарной кучи делается не более двух сравнений. Высота дерева не превышает $\lg(n)$. Значит всего сравнений было сделано не больше $2\lg(n)$. Времени было затрачено $O(\lg(n))$, ведь всего `strengthening_sift_down` делает не более $\lg(n)$ итераций. $\square$

swapping_sift_down

Реализуем, не самую эффективную по времени, зато оптимальную по количеству сравнений, функцию `swapping_sift_down(i, n)`, см. Рис. 2.

```
method parent(i)
  if i = 0
    | return 0
  return  $\lfloor (i - 1) / 2 \rfloor$ 

method bottom-up-search(i, j)
  while j > i and a[j] ≥ a[i]
    | j ← parent(j)
  return j

method swap-subtrees(u, v, n)
  j ← 1
  while v < n
    for i ← 0, 1, ..., j - 1
      | swap(a[u + i], a[v + i])
    u ← left-child(u)
    v ← left-child(v)
    j ← 2 * j

method leftmost-leaf(i, n)
  while left-child(i) < n
    | i ← left-child(i)
  return i

method lift-up(i, j, n)
  x ← a[j]
  a[j] ← a[i]
  while j > i
    swap(a[parent(j)], x)
    if a[sibling(j)] < a[j]
      | swap-subtrees(j, sibling(j), n)
    j ← parent(j)

method swapping-sift-down(i, n)
  k ← leftmost-leaf(i, n)
  k ← bottom-up-search(i, k)
  lift-up(i, k, n)
```

Рис. 2. Реализация функции `swapping_sift_down`.

Она делает следующее: 1) Выбирает самый левый лист; 2) ищет начиная с этого листа (снизу-вверх) куда можно пристроить элемент из корня; 3) опускает корень на подходящую глубину, попутно заботясь о сильном порядке.

При вызове функции `swap_subtrees` произвольная куча может стать невалидной, поэтому мы предполагаем, что наша куча полна [т.е. при высоте h в ней лежит $2^h - 1$ элемент].

Лемма 2 `swapping_sift_down(0, n)` работает за $\lg(n) + O(1)$ сравнений и $O(n)$ перемещений.

Доказательство. Если место было найдено на уровне g [вызовом функции `bottom_up_search`], то потом будет сделано лишь $\lg(n) - g + O(1)$ сравнений при вызове `lift_up`. Так как всего уровней не больше $\lg(n)$ и `swap_subtrees` может отработать на каждом уровне лишь единожды, всего перемещений было сделано не более $\sum_{h=1}^{\lg(n)} 2^h = O(n)$. $\square$

Ленивая куча

Возьмем бинарную кучу и скажем, что иногда в ней может порядок не соблюдаться [такую кучу назовем *ленивой кучей*].

Ленивая куча состоит из трех частей: 1) главной кучи [aka *main heap*]; 2) зоны погружения [aka *submersion area*]; 3) буферной зоны для вставок [aka *insertion buffer*].

Использовать эти части мы будем так. Элементы добавляются только в insertion buffer. При заполнении insertion buffer, половина его элементов переносится в зону погружения. Элементы из зоны погружения являются частью главной кучи, на которых порядок кучи может нарушаться. Пока insertion buffer претерпевает изменения [в него что-то добавляют/из него что-то удаляют], элементы из зоны погружения потихоньку *погружаются* в главную кучу, то есть порядок на них потихоньку становится правильным. В итоге к моменту, когда insertion buffer в очередной раз заполнится или опустеет, элементы из зоны погружения будут полностью встроены в главную кучу [мы должны что-то подобное требовать, ведь если insertion buffer снова заполнился, то нам нужно из него куда-то деть половину элементов, если insertion buffer опустел, то нам нужно что-то удалять откуда-то еще]. Чтобы гарантировать, что зона погружения полностью успевает встроиться в главную кучу, будем поддерживать размер insertion buffer равным $\lg^2(n')$, где n' — размер главной кучи.

Теперь подробнее как все устроено внутри. Пусть $t = \lfloor \lg(1 + \lg(1 + n')) \rfloor$ и $k = 2^{t-2}$. Все расположено в массиве размера $O(n)$. Сначала в нем записана главная куча, после — зона погружения, которая рассматривается как часть главной кучи [то есть в зоне погружения можно обращаться к `parent`, `sibling`, `left-child`, `right-child`]. В самом конце расположен insertion buffer. Insertion buffer состоит из k кусков [aka *chunks*], в каждом из которых помещается k элементов [привет корневой декомпозиции]. На первом месте каждого куска расположен минимальный в нем элемент. Также храним индекс минимума среди минимальных в кусках. В итоге функции `add` [добавление элемента] реализуется за $O(1)$ сравнений и $O(1)$ времени, `extract_min` [удаление минимального элемента] — за $\lg(n) + O(1)$ сравнений и $O(\lg(n))$ времени.

Сначала реализуем погружение за $O^*(1)$ [амортизированное время].

<pre> method sift-down(i, n) x ← a[i] while left-child(i) < n j ← left-child(i) if sibling(j) < n and a[sibling(j)] < a[j] j ← sibling(j) if x ≤ a[j] break a[i] ← a[j] i ← j a[i] ← x </pre>	<pre> method submersion(n', n) r ← n - 1 ℓ ← max{n', parent(r) + 1} while r ≠ 0 ℓ ← parent(ℓ) r ← parent(r) for i ← r, r - 1, ..., ℓ sift-down(i, n) </pre>
--	---

Рис. 3. Реализация функции `submersion`; n' — размер главной кучи, n — размер главной кучи вместе с зоной погружения.

Как это работает? Элементы из зоны погружения, находящиеся где-то внизу главной кучи, постепенно начинают «всплывать» вверх. Порядок между главной кучей и зоной погружения восстанавливается слой за слоем.

Лемма 3 `submersion` работает за $O(\lg^2(n'))$ [с выбранными параметрами это гарантирует время $O^*(1)$].

Доказательство. Так как в зоне погружения содержится не более $\lg(n')^2 \ll n'$ элементов, все они будут лежать в нижнем слое кучи.

Все вычисление разобьем на две фазы: в первой фазе задействованы по меньшей мере два элемента кучи, во второй фазе — не более двух.

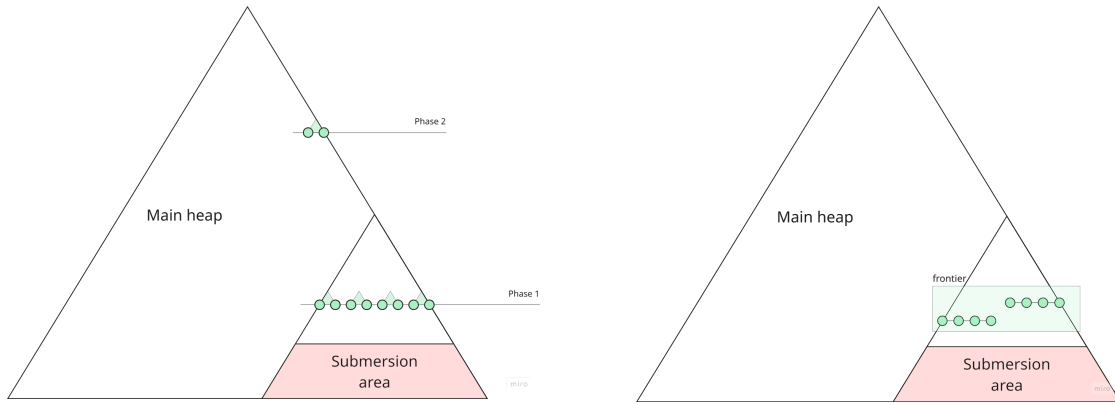


Рис. 4. Разбиение submersion-процесса на фазы (слева), граница (справа).

За сколько обрабатывает первая фаза? На j -ом с конца уровне, будет задействовано не более $\lg^2 \frac{n'}{2^j}$ элементов кучи. Для каждого из них просеивание отработает за $O(j)$. Значит затрачено $O\left(\sum_{j=1}^{\lg^2(n')} \frac{j}{2^j} \lg(n')^2\right) = O(\lg^2(n'))$ времени. За сколько обрабатывает вторая фаза? Всего будет сделано не более $2 \lg(n')$ просеиваний. Каждое из них работает за время $O(\lg(n'))$. Значит на вторую фазу будет потрачено $O(\lg^2(n'))$ времени. В итоге на весь submersion будет затрачено $O(\lg^2(n'))$ времени. $\square$

Мы научились делать push за $O^*(1)$. Но мы не хотим мириться с амортизированным временем. Давайте научимся делать push за $O(1)$.

Будем «деамортизировать» функцию submersion. Каждый раз, когда в кучу поступает запрос push или pop мы будем проделывать какое-то конечное число шагов submersion-процесса.

Как будет реализована деамортизация? Для отслеживания прогресса submersion-процесса мы будем поддерживать два интервала (ℓ_1, r_1) и (ℓ_2, r_2) . Эти два интервала лежат на двух подряд-идущих уровнях, причем $\ell_2 - 1 = \lfloor \frac{r_1 - 1}{2} \rfloor$. Эти два интервала называются границей [aka frontier], см. Рис. 4. В дополнение к границе будем хранить индекс элемента, для которого сейчас выполняется sift_down. При вызове операций push или pop мы проделываем конечное число шагов sift_down для текущего элемента границы. Как только вызов sift_down завершается граница обновляется.

Операции push никак не будут мешать submersion-процессу — элементы просто добавляются в конец insertion buffer. Операции pop уже могут существенно повлиять на submersion-процесс: при вызове pop из кучи должен удалиться минимальный элемент. Минимальный элемент может находиться в одном из трех мест: 1) main heap; 2) insertion buffer; 3) submersion area.

Если минимальный элемент находится в main heap. Тогда минимальный элемент находится в корне главной кучи, достаточно поменять его с последним элементом кучи и вызвать sift_down. sift_down должен остановиться, если встретил границу — так как позже граница будет просеена submersion-процессом.

Если минимальный элемент оказался в insertion buffer, то достаточно вызвать метод extract_min, работа которого была описана выше.

Как быть если минимальный элемент оказался в submersion area? Заметим, что минимальный элемент обязан находиться на границе. Будем поддерживать на границе такую же структуру, как и на insertion buffer: разобьем все на k частей, в каждой по k элементов. Возможно, в первой и последней частях будет меньше k элементов. Для всех частей, кроме первой и последней, на первом месте будет храниться минимальный элемент,

индекс минимального элемента в первой и последней части запомним отдельно. На границе также введем методы `add` и `extract_min` по аналогии с `insertion buffer` [единственно отличие: при обновлении минимального элемента нам придется просеивать его, чтобы не нарушать порядок кучи в уже обработанной части `submersion area`]. В итоге `extract_min` будет нам обходиться в $O(\ln(n))$ времени и $\ln(n) + O(1)$ сравнений.

В итоге получается структура данных, которая 1) обрабатывает `push` за $O(1)$ времени; 2) обрабатывает `pop` за $O(\ln(n))$ времени и $O(\ln(n))$ сравнений. Заметим, однако, что `pop` работает за $\lg(n) + O(1)$, если минимальный элемент находится в `submersion area` или `insertion buffer`.

Сильно-ленивая куча

Сильно-ленивая куча состоит из трех частей: 1) `main heap`, 2) `submersion area`, 3) `insertion buffer`. Назначения этих частей такие же как у ленивой кучи. Однако от ленивой кучи сильно-ленивая куча отличается тем, что `main heap` состоит из двух слоев: `top heap` и `bottom heaps`. `Top heap` является обычной кучей. `Bottom heaps` являются полными сильными кучами, причем их высота равна либо $t - 3$ и $t - 2$, либо $t - 2$ и $t - 1$. `Top heap` отделено от `bottom heaps` границей [aka *border*], чтобы не путать с границей [aka *frontier*] будем переводить `border` как *бордюр*.

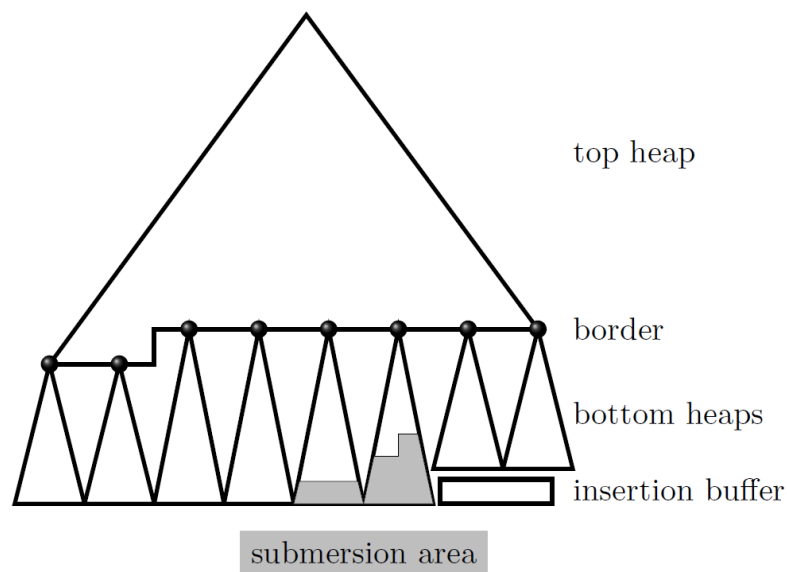


Рис. 5. Схема сильно-ленивой кучи.

У ленивой кучи была проблема: `pop` потреблял $O(\lg(n))$ сравнений, если минимальный элемент лежал внутри `main heap` [так как в обычной куче `sift_down` использует достаточно много сравнений].

Рассмотрим такой следующую функцию `combined_sift_down`, см Рис. 6. Используя ее вместо `sift_down` мы и получим оптимальное количество сравнений на каждую операцию `pop`.

Как работает `combined_sift_down`? Сначала мы хотим понять где будет лежать $a[0]$: он может лежать либо в `top heap`, либо в одной из `bottom heaps`. Для этого мы спускаемся вниз, пока не встретим бордюр. Спускаясь вниз всегда выбираем наименьшего ребенка. Если оказалось, что $a[0]$ должен лежать в `bottom heap`, то вызываем `swapping_sift_down`. Если же $a[0]$ должен

лежать в top heap, то вызываем `binary_search_sift_up`, который находит бинарным поиском место, в которое должен встать $a[0]$ и «прокручивает».

```

method ancestor(i, d)
return  $\lfloor (i+1)/2^d \rfloor - 1$ 

method rotate(i, k, h)
 $x \leftarrow a[i]$ 
for  $d \leftarrow h-1, h-2, \dots, 0$ 
|  $a[\text{ancestor}(k, d+1)] \leftarrow a[\text{ancestor}(k, d)]$ 
 $a[k] \leftarrow x$ 

method correct-place(i, k, h)
 $d \leftarrow h$ 
while  $i \neq k$ 
|  $h' \leftarrow \lfloor (h+1)/2 \rfloor$ 
|  $j \leftarrow \text{ancestor}(k, h')$ 
|  $h \leftarrow h - h'$ 
| if  $a[i] \leq a[j]$ 
| |  $k \leftarrow j$ 
| |  $d \leftarrow d - h'$ 
| else
| |  $i \leftarrow \text{ancestor}(k, h)$ 
return (i, d)

method binary-search-sift-up(i, k, h)
 $(j, d) \leftarrow \text{correct-place}(i, k, h)$ 
rotate(i, j, d)

method combined-sift-down(i, n, h)
 $j \leftarrow i$ 
repeat h times
|  $k \leftarrow \text{left-child}(j)$ 
| if  $a[\text{sibling}(k)] < a[k]$ 
| |  $k \leftarrow \text{sibling}(k)$ 
|  $j \leftarrow k$ 
if  $a[i] \leq a[j]$ 
| binary-search-sift-up(i, parent(j), h-1)
else
| rotate(i, j, h)
| swapping-sift-down(j, n)

```

Рис. 6. Реализация функции `combined_sift_down`.

Лемма 4 `combined_sift_down(0, n)` работает за $O(\lg(n))$ времени и $\lg(n) + O(1)$ сравнений.

Доказательство. Чтобы спуститься до бордюра используется $\lg(n) - \lg(\lg(n)) + O(1)$ сравнений. Если $a[0]$ лежит в bottom heaps, то будет вызвана функция `swapping_sift_down`, работающая за $O(\lg(n))$ времени [так как размер каждой bottom heap $\Theta(\lg(n))$] и $\lg(\lg(n)) + O(1)$ сравнений по Лемма 2. Если же $a[0]$ лежит в top heap, то место, в которое нужно вставить $a[0]$ будет найдено за $\lg(\lg(n)) + O(1)$. Прокрутка же отработает за время $O(\lg(n))$ не используя сравнения вовсе [так как при спуске всегда выбирался наименьший ребенок]. В каждом из случаев получаем $O(\lg(n))$ времени и $\lg(n) + O(1)$ времени. $\square$